Web Development Foundations

Week 4 – JavaScript: "This is Programming"

Day 2: "JS Logic Iteration"

1

Reconnect

Monday:

- variables stored values
- console output made values visible
- `if / else` chose a path

Today:

- the same decision becomes reusable
- functions help organize logic

2

From Monday to Wednesday

Your decision logic becomes a reusable function.

1 Monday: Variables and if/else

```
let score = 84;
let onTime = true;
if (score >= 80 && onTime) {
  console.log("on track");
} else {
  console.log("needs review");
}
```

Decision logic works. But it's written inline.

2 Tuesday: Named function

```
function checkStatus(score, onTime) {
  if (score >= 80 && onTime) {
    console.log("on track");
  } else {
    console.log("needs review");
  }
}
```

The same logic is grouped and named.

3 Wednesday: Function call and console result

```
let score = 84;
let onTime = true;
checkStatus(score, onTime);
// Console output
on track
```

Call the function. Read the result.

Same decision.
Clearer organization.

Reconnect

3

The Working Problem: Logic Gets Repeated Or Messy

Working code can still be hard to follow.

Common signs:

- names are vague
- output is unclear
- the same decision appears more than once
- changing one value is confusing
- the result is hard to explain

4

Working JavaScript Becomes Easier to Follow

Same result. Better structure.

working but hard to follow

```
let s = 84;
let t = true;
if (s >= 80 && t) {
  console.log("on track");
} else {
  console.log("needs review");
}
let s2 = 67;
let t2 = false;
if (s2 >= 80 && t2) {
  console.log("on track");
} else {
  console.log("needs review");
}
```

Same logic repeated.

clearer function

```
function checkStatus(score, onTime) {
  if (score >= 80 && onTime) {
    console.log("on track");
  } else {
    console.log("needs review");
  }
}
let s = 84;
let t = true;
checkStatus(s, t);
let s2 = 67;
let t2 = false;
checkStatus(s2, t2);
```

Named once. Used many times.

Working code can still be improved.
Small changes in structure make code easier to read and reuse.

Messy Logic -> Clearer Logic

5

What Counts as Success Today

- a function has one clear job
- inputs are named clearly
- the function is called
- `return` sends a result back
- console output is explainable

6

Functions Package A Small Job

A function packages steps under a name.

Use a function when:

- a task has a clear purpose
- the logic may be reused
- you want code to read like an idea

Good function names explain the job.

7

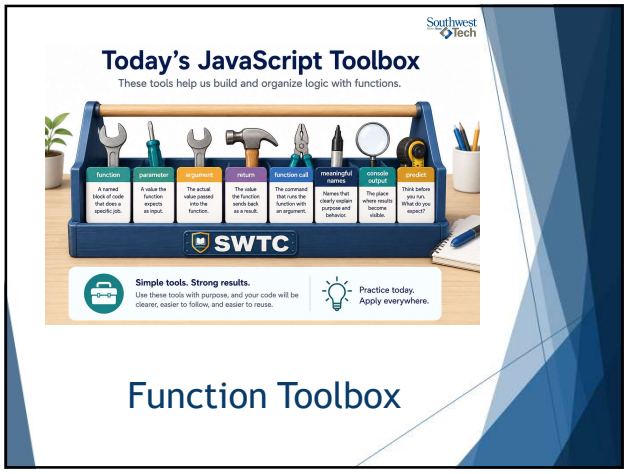
Function Packages A Small Job

8

What We Will Use Today

- function
- parameter
- argument
- `return`
- function call
- meaningful names
- console output
- predict, run, observe

9



Function Toolbox

10

Define Versus Call

Define:

"Here are the steps."

Call:

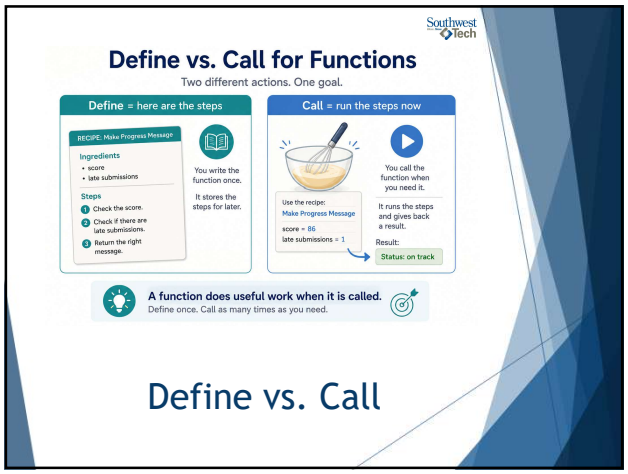
"Run those steps now."

Example:

```
function getProgressMessage(scoreVar, lateSubmissionsVar) {
  // code goes here
}

const message = getProgressMessage(score, lateSubmissions);
```

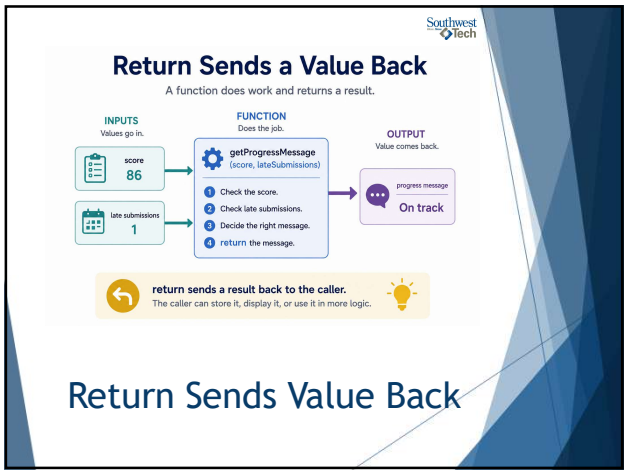
11



Define vs. Call

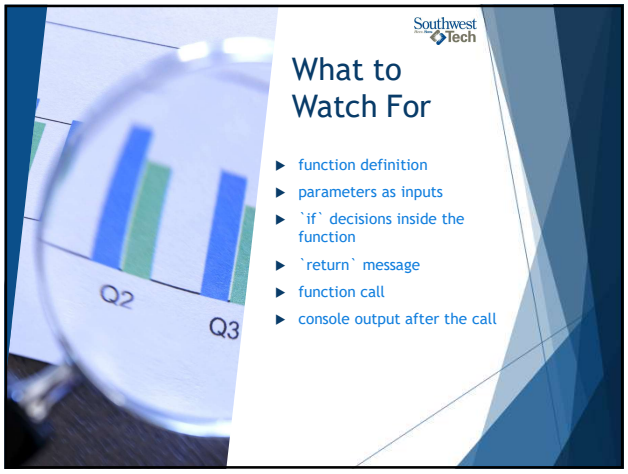
12

13

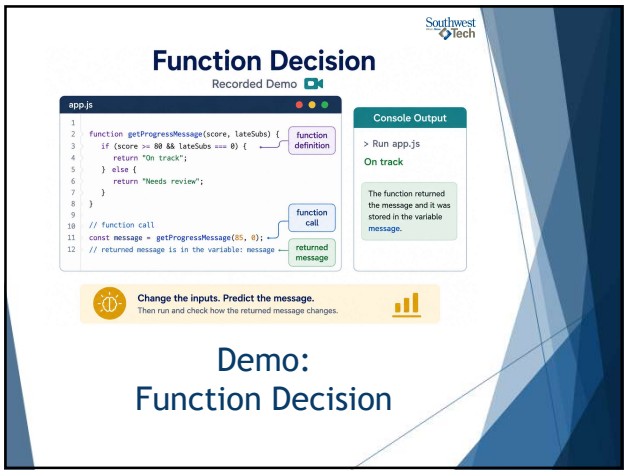


14

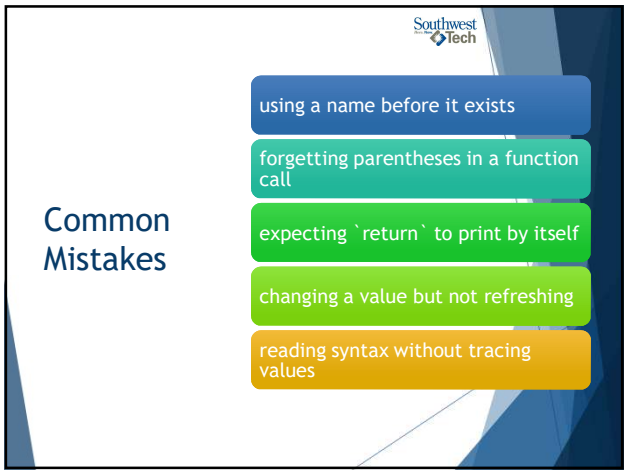
15



16



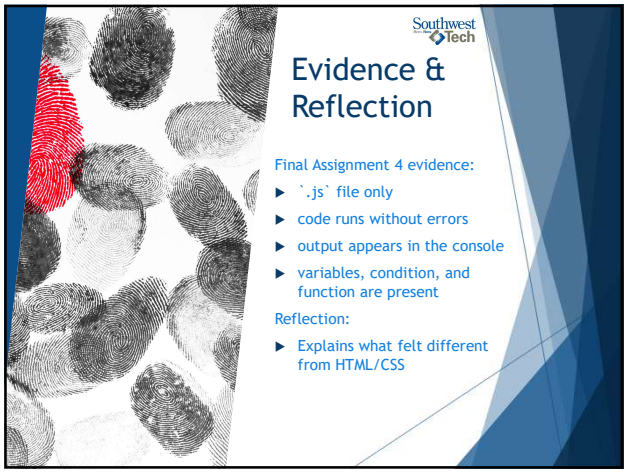
17



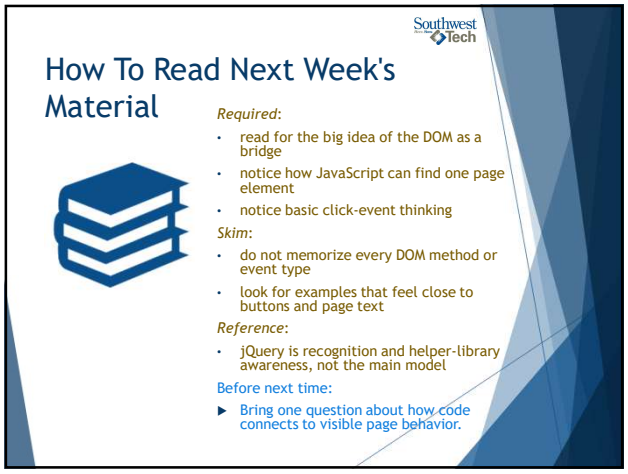
18

19

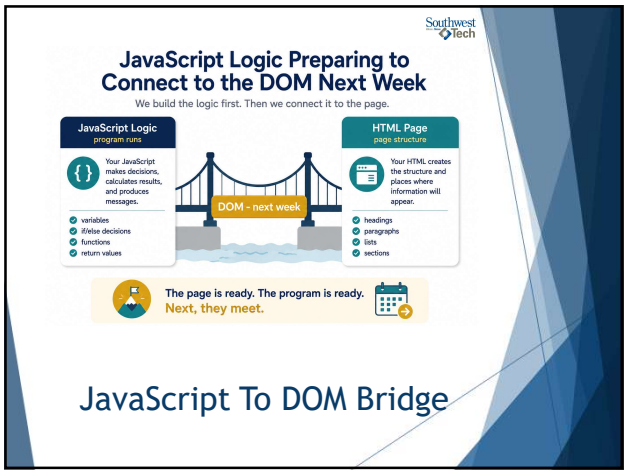
20



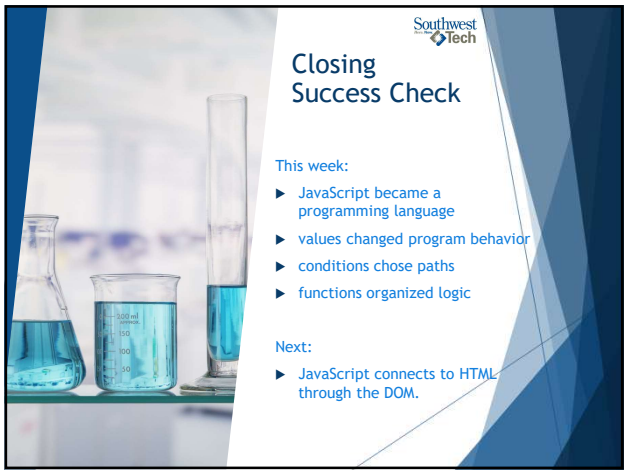
21



22



23



24

25